JETS Federation Agreements For MMS FOM v4.0.0

Table of Contents

1. JETS Overview

JETS, the Joint Emergency Trauma Simulation system, is a digital architecture that enables interoperability between medical simulation systems. The JETS architecture is based on High Level Architecture (HLA), specifically the HLA 1516-2010 standard.

This document describes the agreed upon policies between systems participating in the JETS Federation. The focus of the federation agreements is to provide system developers with an understanding of how to operate in a federation. The Medical Modeling and Simulation (MMS) Federation Object Model (FOM) describes what data and actions (objects and interactions) can be shared in the federation.

Terminology used in the document:

- Run-Time Infrastructure (RTI) – traffic manager for data between systems
- Federate – a system connected to the RTI
- Federation – a group of federates and the RTI

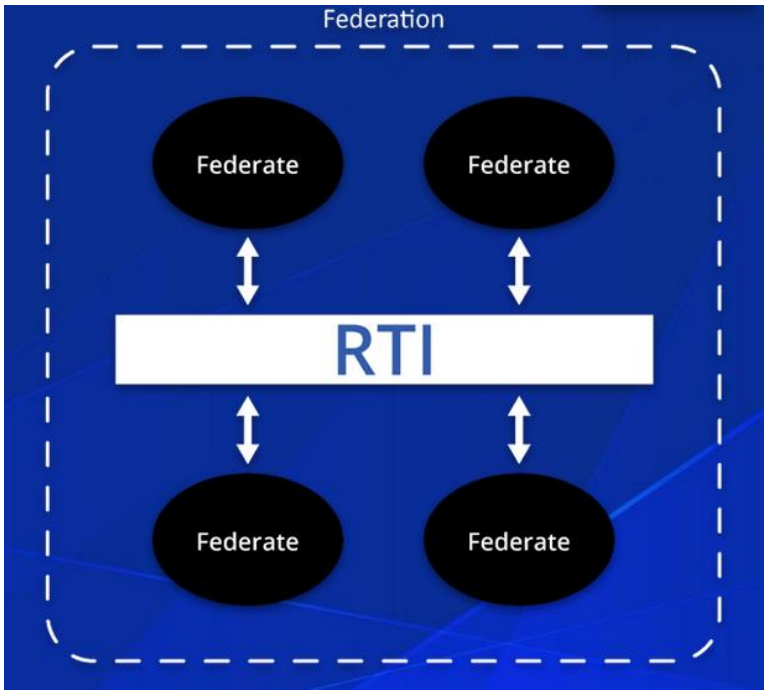


Figure 1. JETS terminology.

2. Federation Object Model

This section of the Federation Agreements describes the contents of the MMS FOM and provides detail on how certain objects interact with others. It is not intended to be a full description of all items in the FOM, but to provide a broad set of guidance of how to read and use the FOM when developing a federate.

2.1. Datatypes and Encoding

The FOM uses naming conventions familiar to most modern language developers. The following tables describe the naming conventions and where they are used in the MMS FOM:

Naming Convention	Description	Example
Upper Camel Case	First letter of each word is capitalized. No spacing between words.	ThisIsAnExample
Lower Camel Case	Initial word is not capitalized. First letter of each other word is capitalized. No spacing between words.	thisIsAnExample
Snake Case	All letters lowercase. Underscores are used to separate words.	This_is_an_example
Uppercase	All letters uppercase. Underscores are used to separate words.	THIS_IS_AN_EXAMPLE

Table 2.1a. Naming convention descriptions.

Datatype	Naming Convention	Example from FOM
Module	Upper Camel Case	SimControl.xml
Object	Upper Camel Case	VitalSigns
Attribute	Lower Camel Case	heartRate
Interaction	Upper Camel Case	SelectScenario
Parameter	Lower Camel Case	patientId
Enumeration Type	Upper Camel Case	MedicationEnum
Enumeration	Lower Camel Case	lactatedRingers
Array	Upper Camel Case	SignsSymptomsArray
Fixed Record	Upper Camel Case	BodyLocationRecord

Table 2.1b. Naming convention examples.

The MMS FOM uses IEEE 1516-2010 predefined data encoding standards. Explicit padding is not required beyond those standards. All datatypes are based on the standard Basic Data types in the IEEE 1516-2010 standard.

- Strings shall be exchanged using the HLAunicode representation.
- Integers and floats shall use Big Endian encoding.

- Enumerated values shall use HLAinteger32BE representation.

2.2. Class Hierarchy and Module Structure

The FOM consist of 8 modules. The following sections describe the purpose and major functionalities of each module.

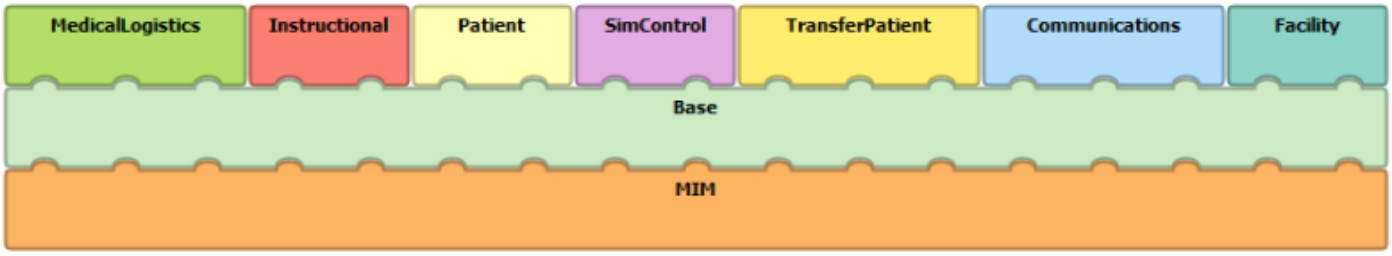


Figure 2.2. MMS FOM module structure.

2.3. Base Module

The Base module includes enumerated and fixed record datatypes that are used by more than one other module. In order to keep modules highly cohesive and loosely coupled, they do not share objects or datatypes between them, with the exception of the Base module. It enables reusability of common components without breaking the FOM design pattern. This includes datatypes such as the BodyLocationRecord and its associated enumeration sets, as this record is shared by many objects across several modules.

2.4. Patient Module

The Patient module contains data representations about a patient's physiology, injuries, treatments, and other patient-focused objects. This module is used to represent the ground truth information about a patient. Multiple patient instances may be created. Each patient instance may have zero or more injuries and zero or more treatments. Patients, injuries, treatments, and other objects in this module are all associated by a unique patientId.

2.4.1. Systems Involved in Patient Module

Types of systems that may be involved in the Patient module are an instructional/control system, a physiology system, and a simulated patient system. More than one instance of each type of system may be involved in the federation, and each system may have one or more federates. These are just three broad categories of systems that are likely to be involved in data from this module.

2.4.1.1. Instructional Systems

An instructional system is involved in managing the scenario, providing instructor control, and monitoring for instructional assessment. Components may include a control system, a learning management system, after action reporting, and scenario management tools. Examples of the functions of this type of system are:

- Initialization of the patient and/or scenario.
- Mapping of clinical concepts to physiological concepts.
- Monitoring federation for learner performance assessment.

2.4.1.2. Physiology Systems

A physiology system is involved in modeling the physiology of the patient and its response to internal and external influences on the patient. The level of fidelity of the physiology modeling is based on the type of physiology used in the system and the requirements of the simulated patients on the federation.

2.4.1.3. Simulated Patient Systems

A simulated patient is anything used to simulate all or part of a patient for the purpose of learner interactions. It can contain one or more components which may be directly or indirectly connected to the federation. One or more simulated patients may exist in the federation, representing one or more patients. Simulated patients can include serious games, manikins, virtual reality patients, part task trainers, standardized patients, or other modalities to represent a patient. In some cases, the simulated patient system and the physiology system may be one in the same.

2.4.2. Patient Model

The patient and interactions between the learner and patient are decomposed into objects based on focus areas. This includes:

- Vital signs (heart rate, respiration rate, blood pressure, etc.)
- Respiratory physiology (tidal volume, total capacity, etc.)
- Signs and symptoms (sounds, ECG rhythms, pain, fatigue, etc.)
- Neurological scales (level of consciousness, Glasgow coma scale, etc.)
- Injuries (acute injuries, type, severity, location, mechanism, etc.)
- Treatments (physical treatments and associated devices, medications, etc.)
- Pathology (underlying or chronic conditions, etc.)
- Personal data (name, weight, age, etc.)
- Various blood and urine labs

- Body fluids (blood volume, urinary output, etc.)

The diagram below describes how the objects within the Patient module combine to create a virtual Patient Model on a federation. The Patient Model has core objects that describe the patient's state, including vital signs, personal data, and labs. The Patient Model can then be enacted on by other objects such as injuries and treatments, which in turn impact the core objects of the Patient Model.

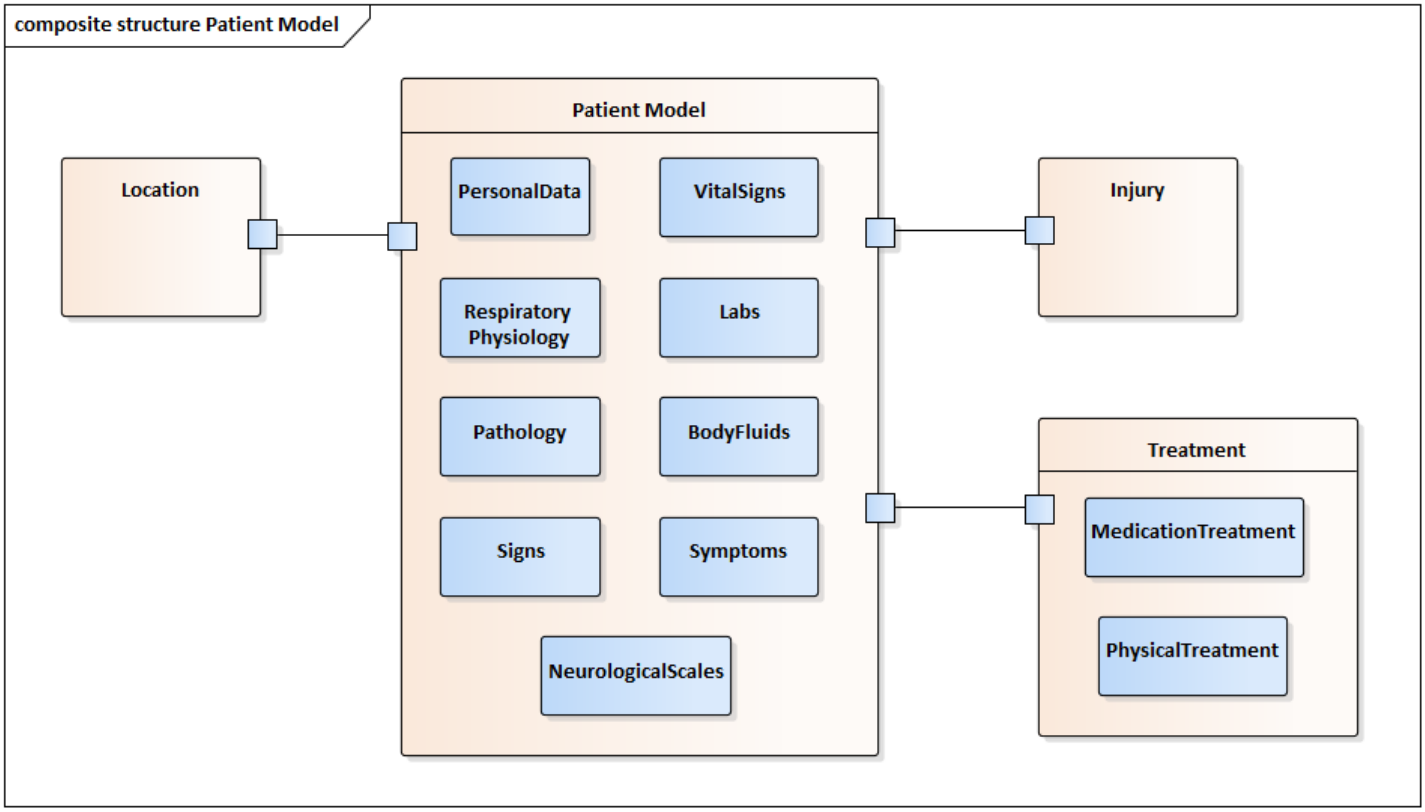


Figure 2.4.2a. Patient Model composite structure.

The conceptual diagram below outlines a sample use case of how the three types of systems may interact on a federation. The directional arrows indicate data that is published (pointing away from the federate) and data that is subscribed (pointing to the federate) by that system. This is not the only use case supported by the federation and is not intended to limit the design of future federations.

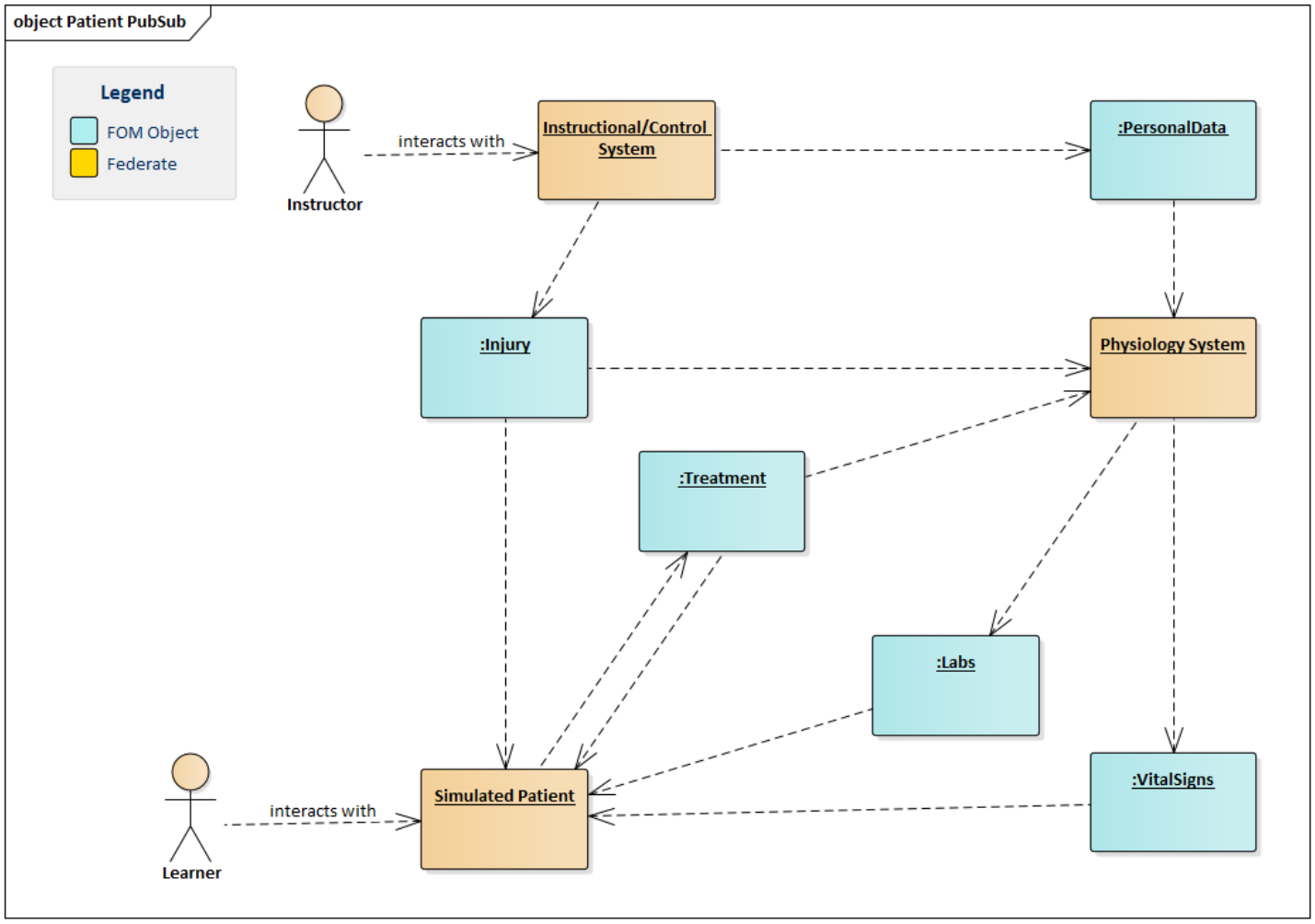


Figure 2.4.2b. Sample publish/subscribe diagram for patient objects.

2.4.3. Treatment and Injury Injection

Before injuries and treatments can be applied, a patient must first exist on the federation. For a patient to exist on the federation, the VitalSigns or PersonalData object must be created to establish the patient model.

Any system on the federation can inflict injuries or apply treatments to the patient model. The system responsible for modeling the patient then reacts to the injuries or treatments and provides updated patient status information.

The primary Treatment object is decomposed into two subclasses: PhysicalTreatment and MedicationTreatment. The parent object is not directly published to the federation, its purpose is to contain common attributes used by the two subclasses. PhysicalTreatment is used for interventions such as stopping hemorrhage and opening the airway and includes both a treatment type and treatment device attribute.

MedicationTreatment is used for applying medications and includes attributes to account for dosages and administration sites.

2.4.3.1. Using the BodyLocationRecord

Within the Patient module, the BodyLocationRecord datatype is used in many of the objects. The BodyLocationRecord and its associated attributes and enumeration sets are contained in the Base module. This record is designed to be based on the Foundational Model of Anatomy (FMA), an ontological standard for defining human anatomy. Rather than listing all unique combinations (e.g. right arm, left leg), the record provides categories including general regions (arm, leg, etc.), three anatomical planes (up/down, left/right, front/back), and more detailed sub-regions (skin, musculature, etc.) that can be combined as needed to support the scenario.

When using the BodyLocationRecord, one or more of the fields may be used to define the location. However, if one field is used, all other fields must be set to notApplicable (rather than null).

The record also includes an attribute to enter an FMA ID. FMA provides a unique numerical identifier for each specific anatomical location. While the MMS FOM does not contain all possible combinations and does not inherently know what the FMA IDs are for each combination allowed in the record, this field can be used for communication between two systems that have knowledge of the FMA database. Some example mappings are listed in the table below.

BodyLocationRecord Combination	FMA Name	FMA ID
upperArm	Arm	24890
upperArm, left	Left Arm	24896
upperArm, left, anterior	Anterior Part of Arm	24898
upperArm, left, anterior, skin	Skin of Anterior Part of Arm	38238

Table 2.4.3.1. Body location FMAID mapping examples.

2.4.4. Physiology Engine Control

Once the federation begins, there may be cases where there is a need to pause the federate modeling the physiology without pausing or stopping all other federates. The PausePhysiologyEngine and ResumePhysiologyEngine interactions can be used for this purpose. Systems can also use the StartPatient, StopPatient, PausePatient, and ResumePatient interactions can be used to more broadly control the patient independently of the rest of the federates.

2.5. Instructional Module

The Instructional module connects the instructor, learner, and assessment tools to the training simulation. This module contains objects that define the learner, scenario, and instructor on the federation. It also includes an Event object which is used to capture detail about the learner that is not captured by objects in the Patient

module. For example, if a learner initiated a treatment on the patient, that data would be captured by the Treatment object; if a learner put on gloves or moved the patient to cover, that data would be captured by the Event object.

2.6. Facility Module

In MMS FOM v3.0.0, this module is largely a placeholder module. It is designed to contain data regarding various types of facilities and their respective patient capacities. The main Facility object contains the core set of attributes, with subclasses used to define specific facility types such as air and ground ambulances. It also contains interactions to control vehicle movement.

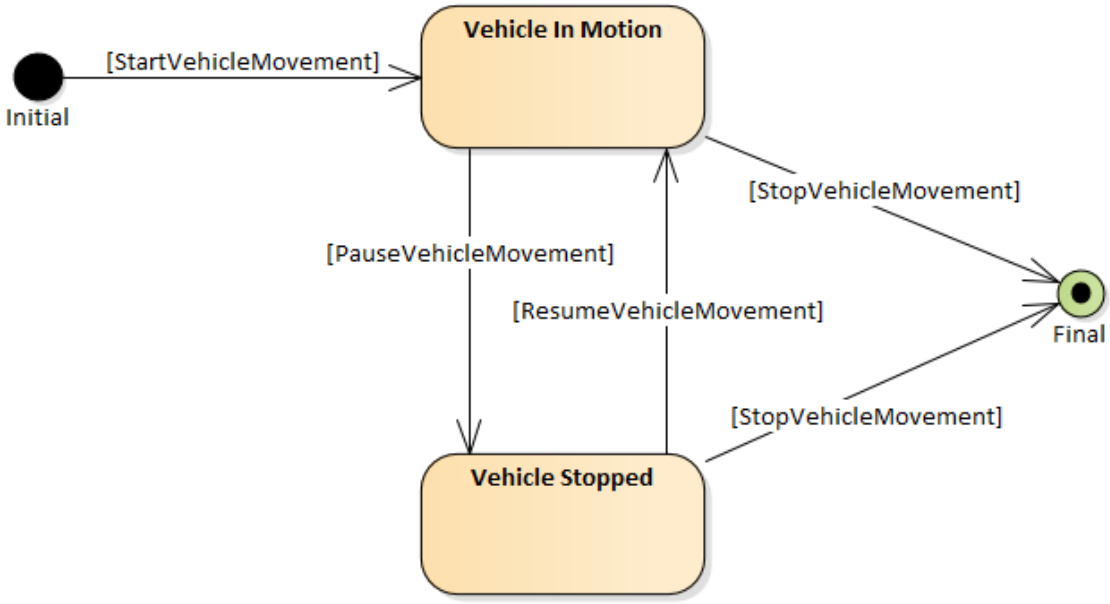


Figure 2.6. Vehicle control states.

2.7. Transfer Patient Module

The Transfer Patient module includes the objects and interactions necessary to model patient evacuation. The objects in this module are currently placeholders for future expansion. Currently, the focus of this module is on the MEDEVAC interactions including the initial request and various types of responses throughout the MEDEVAC process. Note that this is a simulator-to-simulator request for MEDEVAC, not the 9-line request that would be initiated by the learner. The diagram below outlines the expected flow of interactions.

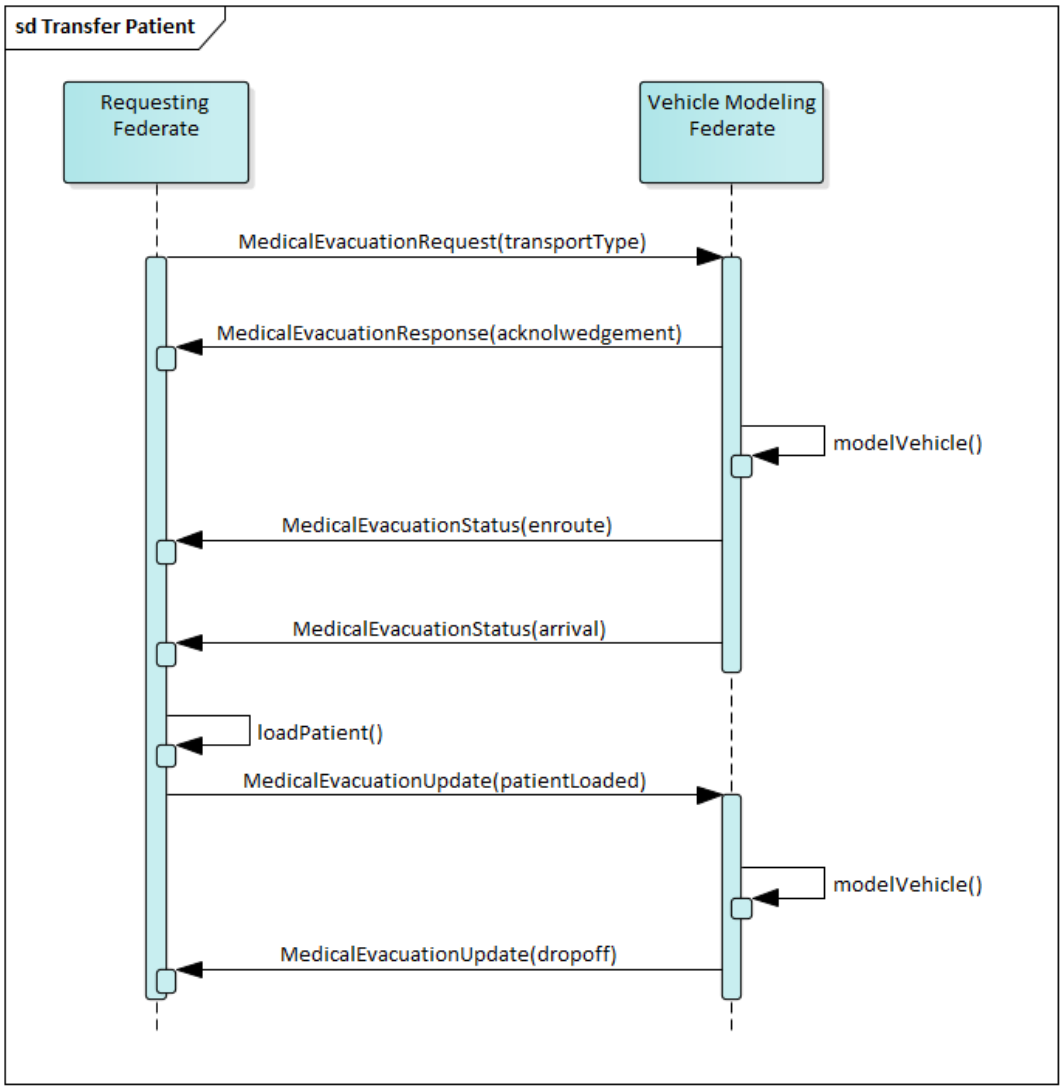


Figure 2.7. MEDEVAC request messaging sequence.

2.8. Medical Logistics Module

The Medical Logistics module contains the representations related to storage, consumption, and transportation of medical supplies. The data in this module is focused on interacting with a logistics simulation system. This module can be used to define product size and weight, logistics transportation routes, and facility logistics requirements (fuel, energy requirements, etc.). The diagram below outlines the conceptual relationship between the objects in this module.

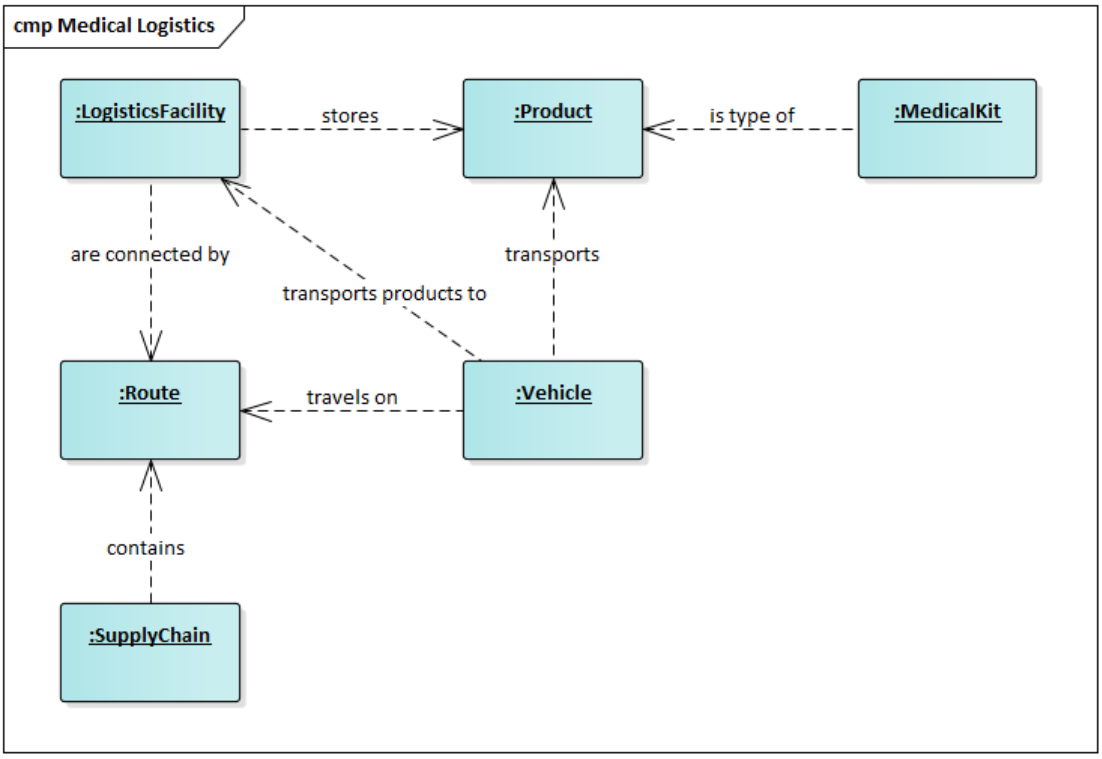


Figure 2.8. Medical logistics object relationships.

2.9. Communications Module

The Communications module is used to capture data on patient documentation forms, radio communications, and command and control messages. As of MMS FOM v3.0.0, only some of these objects and interactions are populated while a majority are placeholders for future expansion. Currently, this module can be used to transfer data on a DD1380 form, a DA4700 form, a Prolonged Field Care Card, a 9-line MEDEVAC request, and a MIST report. Using these objects and interactions assumes that the producing and receiving federates are capable of digital representations of these communications.

This module contains several attributes that will seem to overlap with attributes in the Patient module. The difference is that while the Patient module objects/attributes are focused on the ground truth of the patient, the Communications module is focused on the perceived truth. For example, the Patient module represents the patient's actual heart rate as determined by the patient model; the Communications module represents the value of the heart rate as measured by the learner. These two values may not always be the same.

The module also include a generic Document attribute, which is used to transmit documents as a byte array on the federation.

2.10. Simulation Control (SimControl) Module

The Simulation Control module includes data and commands monitoring the status of and controlling participating simulations. This module includes federation-wide control interactions, a DateTime object to centralize time control on a federation, and FederationState enumerations that allow late-joining federates to know the current operational state of the federation.

3. Federation Execution

This section contains guidance on the general operation and policies of a federation. It is not a detailed user manual for specific federate operations, nor does it provide specific detail on all possible use cases of the federation.

This section uses the term “shall” to indicate policies and actions that federates must comply with to participate in a JETS federation. It uses “may” to indicate policies and actions that are recommended but not mandatory.

During R&D, some rules are relaxed to enable more rapid testing and demonstrations.

3.1. Federation Name

Most RTIs can support multiple federations on the same network. Federates determine which federation to join by its federation name. For the JETS federation, the default federation name is JETS. Federates may assume they will be connecting with the default name if not instructed otherwise. All federates shall be able to be configured to join using a federation name provided at runtime.

The federation name is typically set in the FederateConfig.txt file.

3.2. Federate Name

All federates on a single federation shall have unique federate names. Multiple instances of the same system may be joined to the same federation so long as each instance has a unique federate name (e.g. TestFed1, TestFed2).

The federate name is typically set in the FederateConfig.txt file. If the JETS Developer Package API is being used, the federate name is set in the joinFederation mutation. Systems accessing federation data via the API will not have unique federate names, as the Developer Package is the federate. The diagram below describes this setup.

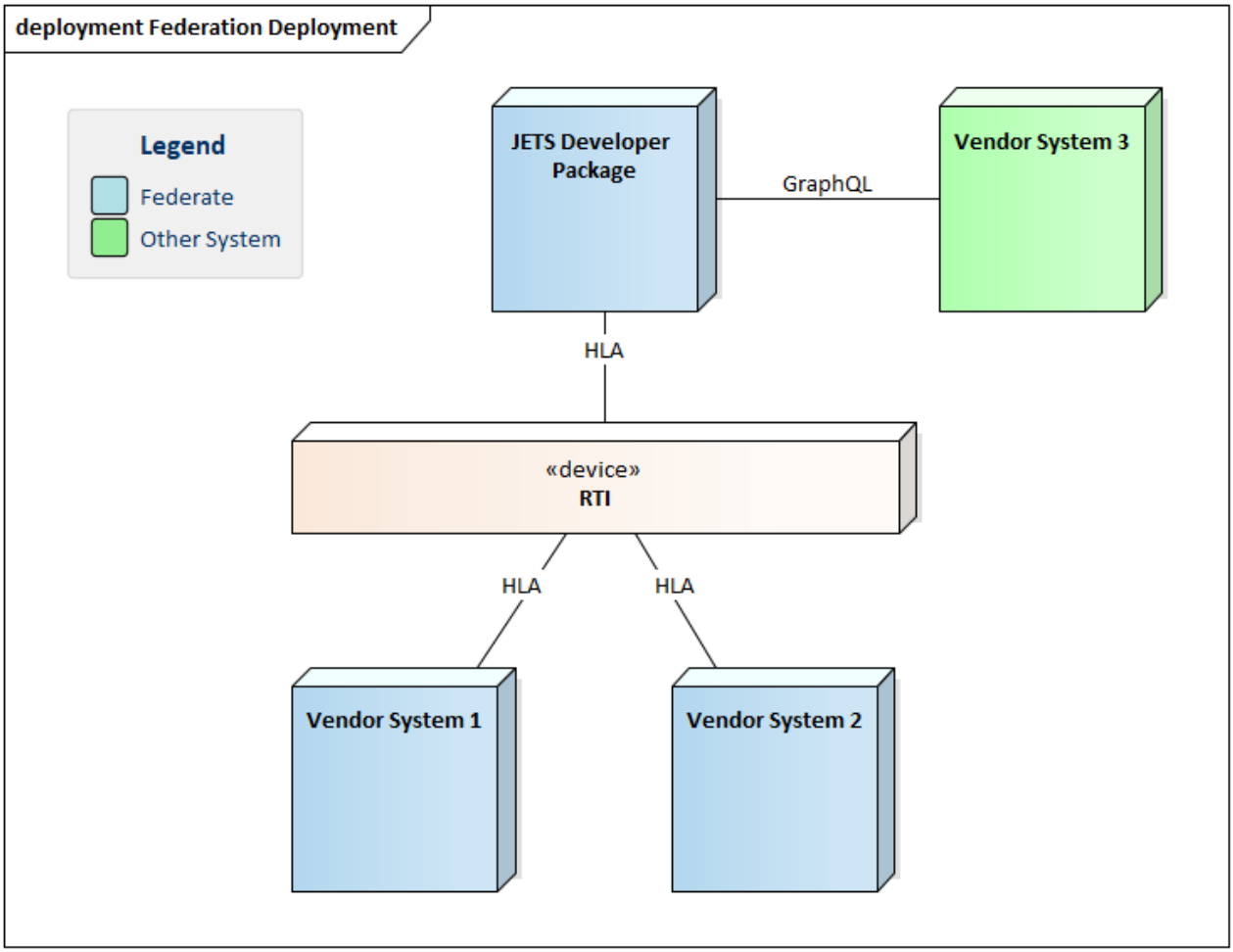


Figure 3.2. Federation deployment diagram.

3.3. Creating a Federation

A federation is created by the first federate that requests to join a federation, with a federation name that is not currently in operation.

3.4. Joining and Resigning from the Federation

There is no specific join or resign order for the JETS federation. Upon joining the federation, a federate shall verify the current state of the federation and respond appropriately. The current state is provided by the FederationState.state attribute.

To close a federation, all federates must resign from the federation. When a federation is closed, all data transmitted on that federation is lost, unless a federate stored the data locally.

3.5. Re-Joining or Late Joining the Federation

If a federate joins the JETS federation after the federation is running, then it is responsible for any gap between its current modeling state and the federation's execution time. The federation shall provide the current state of all objects which the federate is interested in.

3.6. Federation Control

NOTE: while JETS is still in development, the policies in Section 3.6 are not enforced for demonstration purposes.

3.6.1. Federation States

After the JETS federation is created, it will always exist in one of the following states:

- Stopped: no simulation or modeling is occurring. Time is at 0.
- Running: federates are actively executing and time is advancing.
- Paused: simulation and modeling are on hold. Time is not advancing but remains at last point prior to being paused.

The current state of the federation is provided in the FederationState.state attribute. The federation can be transitioned from one state to another using the simulation control interactions in the SimControl module. The following state machine illustrates the federation states and how the interactions are used to transition between them.

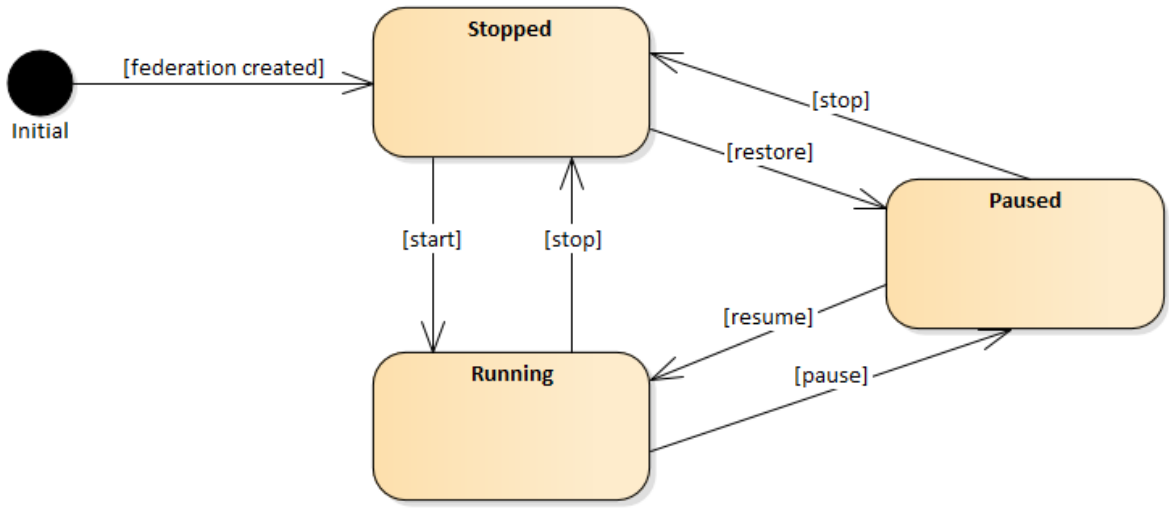


Figure 3.6.1. Federation state diagram.

3.6.2. Federation Control Interactions

Federates shall subscribe and respond to the simulation control interactions: start, stop, pause, resume, save, and restore. Each of these interactions is described below.

- Start: The Start interaction is used to begin the execution of the scenario.
    - During initialization, federates will join the federation and initialize any classes required for starting the federation, but will not begin execution until they receive the Start command.
- Stop: The Stop interaction is used to immediately end of the execution of the federation. The simulationElapsedTime counter will be reset to zero.
- Pause: The Pause interaction stops the simulationElapsedTime counter. Federates shall cease execution but hold in a state to resume execution later.
- Resume: The Resume interaction follows a Pause interaction to indicate that federates shall begin execution again. The simulationElapsedTime counter will continue incrementing.
- Save: The Save interaction signals a federate to save their state in a form that the federate can Restore from. It should only be sent when the federate is in a "Paused" state. The Save interaction will contain a "label" parameter supplied by the publishing federate.
- Restore: The Restore interaction will include a "label" parameter to indicate which save to restore from. The result of the Restore interaction is that the federation is restored to the point in execution when the Save was completed. Upon Restore, federates shall wait in a "Paused" state until they receive a Resume interaction.
- Set Time Scale: Sets the simulation time scale attribute of the DateTime instance. Used to tell the simulation to execute faster than real time.

3.6.3. Publishing FederationStatus

IVIR will provide a MMS Control federate which publishes updates to the FederationStatus object based on the federation control interactions. If the MMS Control federate is not used, then another federate shall be responsible for ensuring the federation state is maintained and complies with the provided state diagram in Section 3.6.1.

3.7. Federation Time

NOTE: while JETS is still in development, the policies in Section 3.7 are not enforced for demonstration purposes.

The concept of time in modeling and simulation differs based on perspective and context of the policies set. Therefore, this Federation Agreements document uses the following definitions for time:

- Real-world time: The time as defined by a chronometer. This can be a wristwatch, a wall clock, or an atomic clock. The rate for real-world time does not change, it always remains at one second per second.
- Simulation time: The time described by the simulated world. Simulation time can be controlled. It can be paused and reset. Its rate can be changed to be faster or slower than real-world time.
- Elapsed time: The amount of time that has passed since a specific event.
- Epoch: Epoch is an event. In the JETS federation, epoch is always defined as Midnight (00:00) on 01 January 1970 UTC.
- Date-time: The representation of time that includes the date and time.

3.7.1. Accessing Time

The MMS FOM contains a DateTime object that contains 4 values for time:

- Simulation Elapsed Time (SET): The amount of time that has elapsed since the beginning of the execution.
- Simulation Date-Time (SDT): The date and time the scenario is modeling, not necessarily the same as real-world time.
- Current Date-Time: The real-world time in the date-time format in UTC.
- Time Scale: The time scale at which the simulation time is running compared to real world time (e.g. 2:1 indicates the simulation is running 2 seconds for every 1 real world second).

Federates shall subscribe to the DateTime object and use the values required by the scenario.

3.7.2. Federate Time Responsibilities

IVIR will provide a MMS Control federate that is responsible for maintaining the 4 perspectives of time as determined by scenario requirements. If the MMS Control federate is not used, then another federate must assume the responsibility for publishing the DateTime object.

3.7.3. Time Representation in the FOM

The MMS FOM uses both SET and SDT in attributes and parameters. A federate can determine which version is expected by the naming convention used.

- Any attribute/parameter that ends with “Time” is expecting Simulation Elapsed Time.
- Any attribute/parameter that ends with “DateTime” is expecting Simulation Date Time.

In some cases, attributes may include the word “time” but not follow the above conventions. In some cases, it is in order to represent time in a specific format for the purpose of accurately representing the model. For

example, the forms in the Communications module include attributes that represent time in a string format for the purpose of capturing a manually written time that may not comply with any standards or format. All such exceptions are captured in the attribute's semantics.

3.8. Publishing and Subscribing

3.8.1. Object Update Policy

All object attributes shall update on change. Attributes that do not change shall not be updated. The rate of updates shall be determined by the publishing federate.

3.8.2. Object IDs

The owning federate is responsible for ensuring the uniqueness of object IDs within objects of the same type. Object IDs, such as Patient.patientId, may be used to link one object type to another (VitalSigns.patientId). The federation does not enforce uniqueness of such values. In a case where the federate has a need to track a specific object, then it may use its HLA instance name to guarantee the exact object is being accessed.

3.8.3. HLA Instance Names

It is the responsibility of each federate to ensure each HLA object published uses a unique HLA instance name. To ensure uniqueness, federates may use the following name pattern:

```
<federate name>/<object name>/<local id>
```

where

- <federate name> is the name of the producing federate
- <object name> is the object named as defined in the FOM
- <local id> is any string unique to each object instance in this federation

For example, a simulated patient federate with the federate name of FemaleManikin would publish its third treatment with the HLA instance name of `FemaleManikin/PhysicalTreatment/03`.

3.9. Federation Modeling

3.9.1. Coordinate System

The coordinate system used in the JETS federation is decimal degrees of latitude and longitude, with altitude captured as feet above mean sea level.

3.9.2. Dead Reckoning

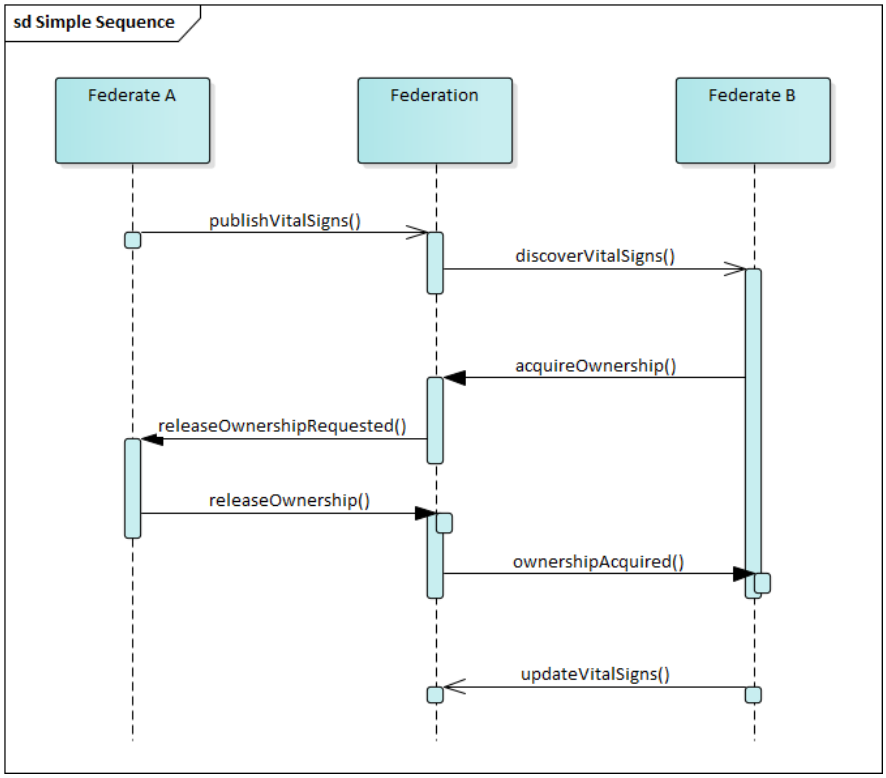
The JETS federation does not include any data types that require dead reckoning.

3.9.3. DDM/Interest Management

The JETS federation does not currently support Data Distribution Management (DDM) or interest management.

3.9.4. Control Transfers

Federates can transfer ownership of an object to other federates. The diagram below describes the process to transfer ownership. In DevPackage, two mutations have been added to support this: vitalSignsAcquire and vitalSignsRelease. Additionally, the VitalSigns object has a new attribute to describe its ownership state.



IVIR INC.

jets@ivirinc.com